



GUI设计与事件处理

余 烨

本章学习提示

- 本章学习Java语言的用户界面的设计，及如何让用户和程序进行友好方便的交互。
  - AWT、Swing包中的组件
  - 布局管理器的概念和使用
  - Java的事件处理机制

目　录

1 AWT组件

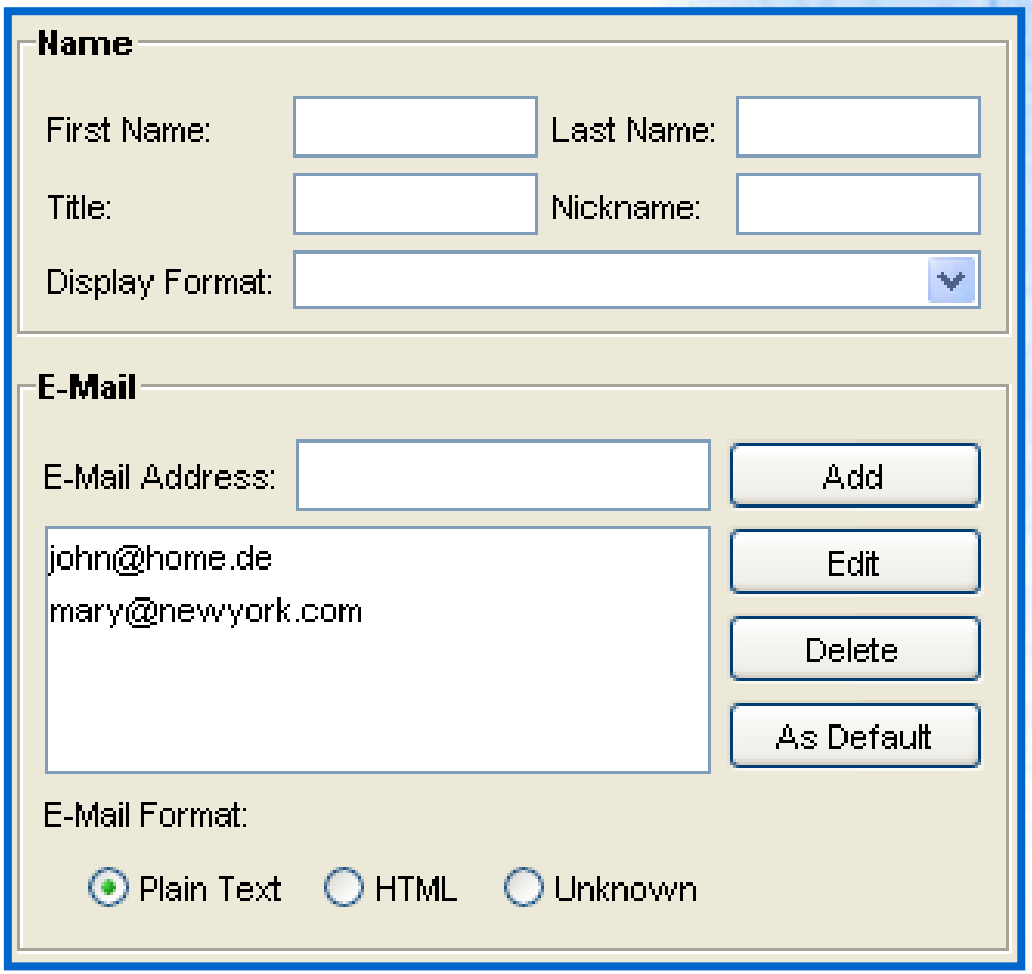
2 布局管理器

3 Java的事件处理

4 Swing包

GUI

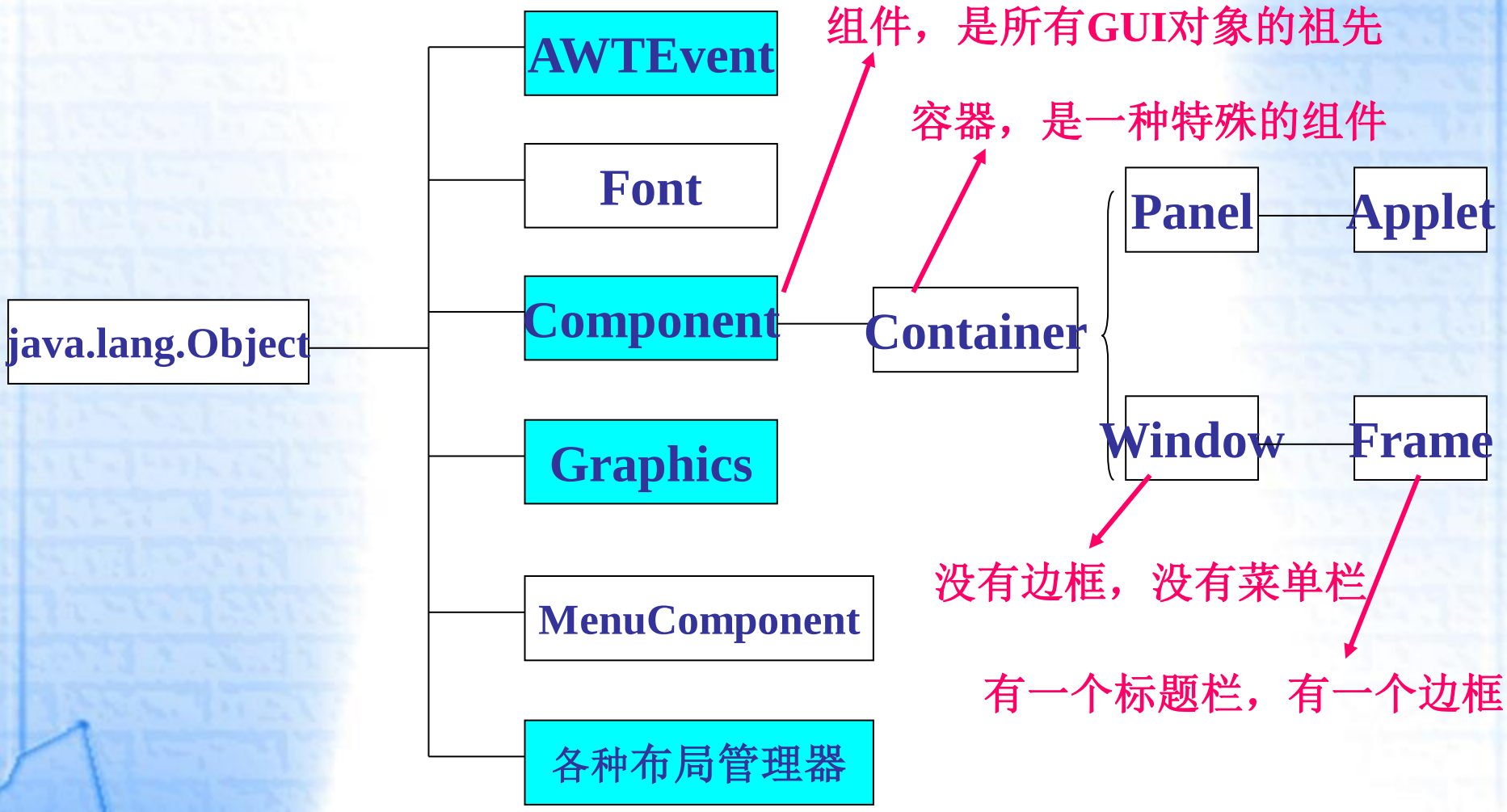
- Graphical User Interface,
  图形用户界面
  - 包括菜单、按钮、工具栏等各种元素
  - 通过提供菜单、按钮等，完成对计算机发出指令、启动应用程序等操作任务



Java 的图形用户界面AWT包

- JDK中提供了AWT和Swing两个包，用于GUI程序的设计和开发
- AWT：Abstract Window Toolkit，位于java.awt包中
- 在java中，AWT是用来处理图形最基本的方式，它可以用来创建java的applet和窗口。AWT包提供：
  - 组件：如：窗口、按钮、菜单等
  - 容器：是GUI元素的容器。实现组件管理、布局管理
  - 布局管理器：用来安排图形界面组件的位置
  - Graphics：在组件上进行图形绘制所使用的图形环境的父类
  - 事件处理对象：用来处理图形界面组件所触发的事件

java.awt包


java.lang.Object
AWTEvent
Font
Component
Graphics
MenuComponent
各种布局管理器
组件，是所有GUI对象的祖先
Container
容器，是一种特殊的组件
Panel
Applet
Window
Frame
没有边框，没有菜单栏
有一个标题栏，有一个边框

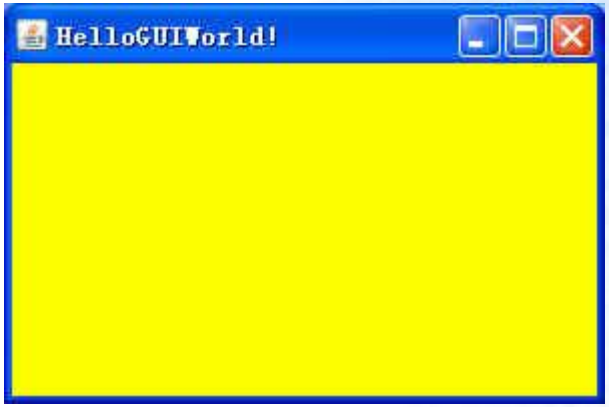
建立窗口的Frame类

- Java 中“窗口”被视为一个容器。它可以把各种不同的图形界面组件放置到这个容器中，而这些图形界面组件就是容器中的接口，通过图形界面组件所提供的方法，来完成一定的功能。
- 创建窗口的基本和必要操作包括如下步骤：
  1. 给窗口一个标题
  2. 设置窗口的大小、背景颜色
  3. 在屏幕的某个位置放置窗口
  4. 显示窗口

MyFrameTest.java

显示窗体之例

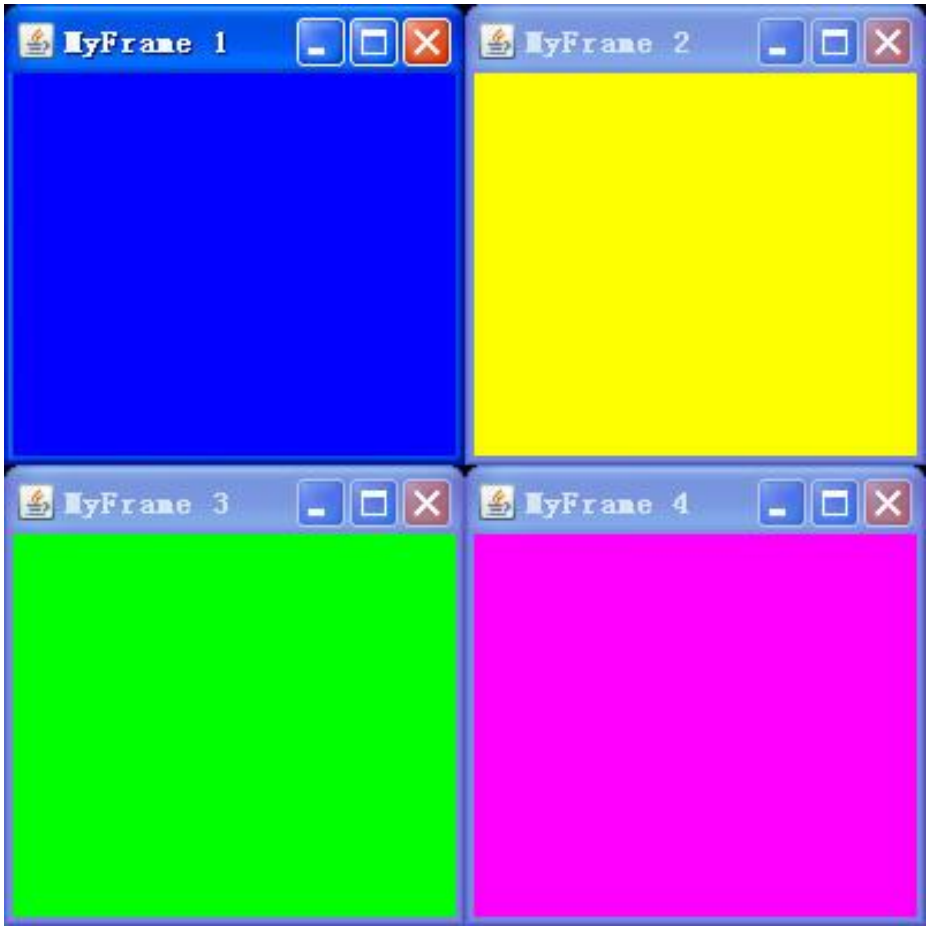
```
//show GUI Frame
import java.awt.*;
class MyFrame extends Frame
 {    //新建的窗口类
      MyFrame(String s)
       { super(s);
         this.setSize(300,200);
         this.setBackground(Color.yellow);
         this.setVisible(true);
       } }
public class MyFrameTest
{  public static void main (String[] args)
    {  MyFrame f=new MyFrame("HelloGUIWorld!");
    }  }
```



Frame类

- 框架(**Frame**)
  - 构造方法: Frame(String title)
  - 一个Frame被创建后初始大小为(0,0)，需要设置大小
- 对于Frame外观的操作比较重要的方法如下：
  - setTitle——设置窗口中标题栏的文字。
  - setResizable——设置用户是否可以改变框架大小。
  - dispose ——关闭窗口，并回收该窗口的所有资源。
  - setSize——设置组件的大小。
  - setBackground——设置组件的背景颜色。
  - setVisible——设置组件是否可见。
  - setBounds——重新设置组件的大小和位置。

Frame窗口例



- 例 TestMultiFrame.java

构建MyFrame类

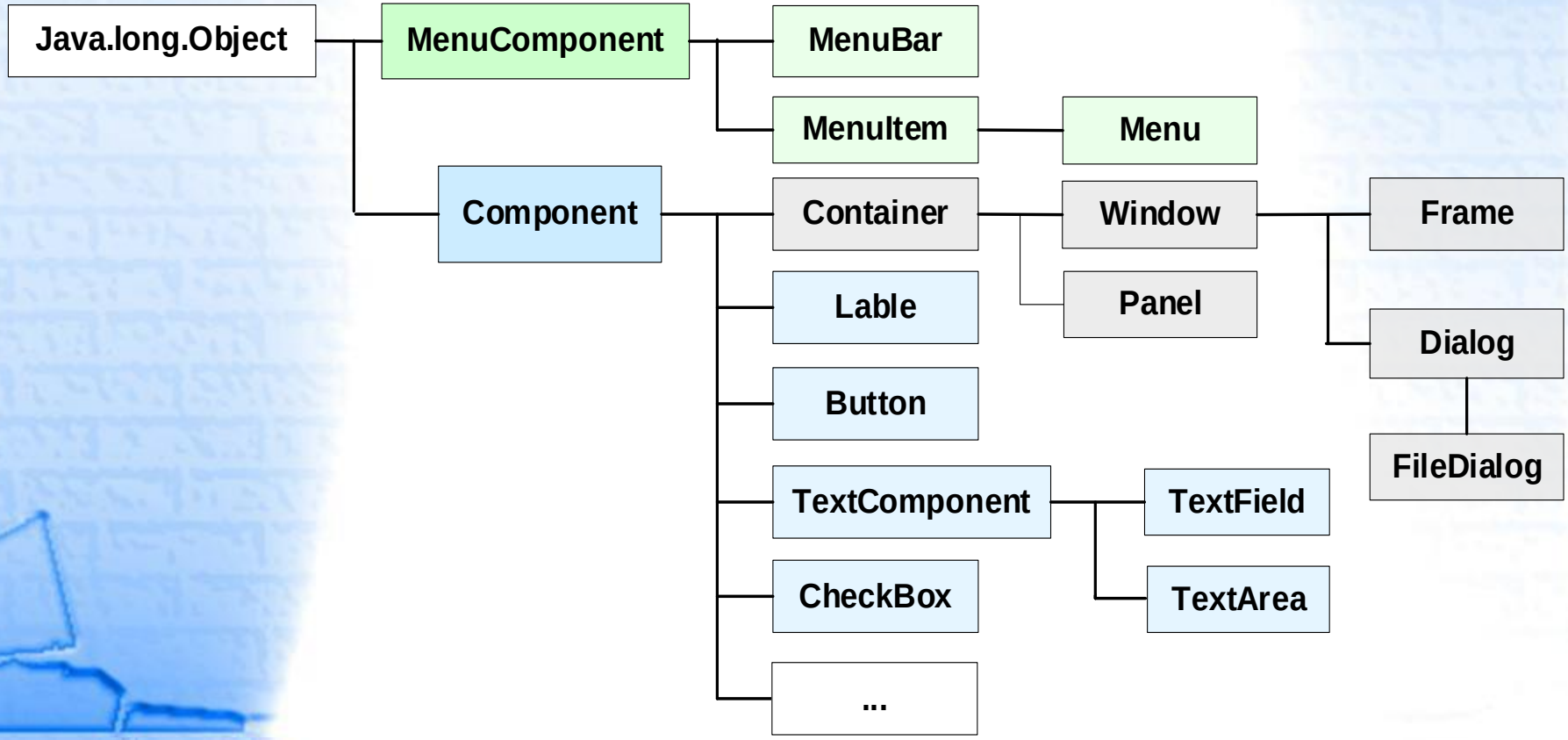
```
class MyFrame extends Frame
{
        static int id = 0;
        MyFrame(int x, int y, int w, int h, Color myColor)
         {
                super("MyFrame " + (++id));
                this.setBackground(myColor);
                this.setLayout(null);
                this.setBounds(x, y, w, h);
                this.setVisible(true);
         }
}
```

创建窗口

```
public class TestMultiFrame
{
    public static void main(String args[]){
        MyFrame frm1 = new MyFrame(100, 100, 200,
200, Color.blue);
        MyFrame frm2 = new MyFrame(300, 100, 200, 200,
Color.yellow);
        MyFrame frm3 = new MyFrame(100, 300, 200, 200,
Color.green);
        MyFrame frm4 = new MyFrame(300, 300, 200, 200,
Color.magenta);
        }
}
```

AWT组件库

- 标签 按钮 单行文本 多行文本 单选框 复选框
- 菜单 下拉菜单
- 对话框 文件对话框



常用AWT组件

- 标签 (Label) - Java.awt.Label
  - 在界面上显示一行文字。

    Label label = new Label("Hello World!")。

    void setText(String s);

    String getText();
  - 一般与事件无关
- 按钮 (Button)

  Button b = new Button("Help");
  - 点击按钮会激发ActionEvent事件。
  - 使用ActionListener接口监听、处理该事件
- 单行文本 (TextField)
  - 用来接受用户输入

    TextField t = new TextField("name", 10);
  - setEnabled(false)设置为只读。
  - 在TextField处回车会激发ActionEvent事件
  - 使用ActionListener接口监听、处理该事件

- **多行文本 (TextArea)**
  - 用来接受用户输入的多行文本
  - TextArea t = new TextArea(“name”, 5， 50);
  - setEnabled(false)设置为只读
  - 在TextField处回车表示换行，不会激发ActionEvent事件
- **复选框 (Checkbox)**
  - 提供简单的on/off开关
  - Checkbox c1 = new Checkbox(“Red”,null, true);
  - setEnabled(false)设置为只读
  - 当复选框发生改变时，会激发ItemEvent事件
- **单选框 (CheckboxGroup)**
  - 提供单项选择。每个选项是相关联、互斥的
  - 首先构造一个**CheckboxGroup**对象，然后为这个Group添加Checkbox
  - setEnabled(false)设置为只读
  - 当复选框发生改变时，会激发ItemEvent事件

- **对话框 (Dialog)**
  - 可以实现用户数据的接收，提示一些信息等
  - 可分为模式和非模式对话框
  - Dialog d = new Dialog(parentFrame, “Title” , false);
  - setVisible(boolean)来控制对话框的可见性
- **文件对话框 (FileDialog)**
  - 用来为用户选择文件，是模式对话框
  - FileDialog fd = new FileDialog(parentFrame, “Title”);
  - setVisible(boolean)来控制对话框的可见性
  - String fileName = fd.getFile();
  - 一般与事件无关
- **下拉列表 (Choice)**
  - Choice ColorChooser = new Choice();
  - ColorChooser.add("Green");
  - ColorChooser.add("Red");



示例

- 在空的窗体上增加标签、按钮和单选框



Example01.java

目　录

1 AWT组件

2 布局管理器

3 Java的事件处理

4 Swing包

界面布局管理器

- 当把组件添加到容器中时，希望控制组件在容器中的位置。
- Java.awt包中共定义了5种布局类，每种布局都对应一种布局策略，分别是：
  - FlowLayout (顺序布局)
  - BorderLayout (边界布局)
  - CardLayout (卡片布局)
  - GridLayout (网格布局)
  - GridBagLayout (复杂网格布局)
- 容器可以使用方法：　setLayout(布局对象);

  来设置自己的布局

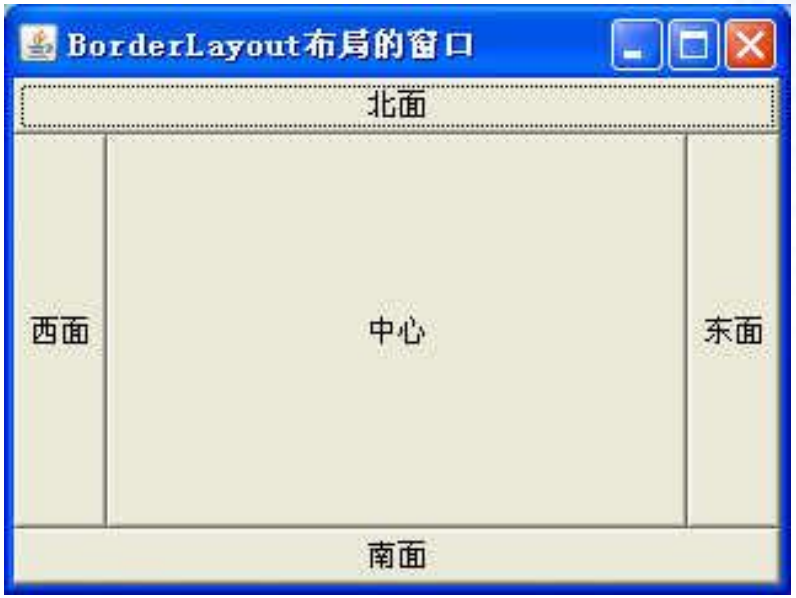
FlowLayout

- FlowLayout的布局策略是将容器中的组件按照加入的先后顺序从左向右排列。如果一行排满转下一行继续，每行均采取居中排列。
- **是Panel型容器的默认布局**，即Panel及其子类创建的容器对象，如果不专门为其指定布局，则它们的布局就是FlowLayout型布局。
- 例 FlowLayoutTest.java



BorderLayout

- BorderLayout布局策略是把容器内的空间划分为东、西、南、北、中5个区域分别用英文的East、West、South、North、Center表示，向容器中加入每个组件都要指明在容器的区域。
- 是Window型容器的默认布局，例如Frame、Dialog都是Window类的子类，它们的默认布局都是BorderLayout 布局。
- BorderLayoutTest.java



CardLayout

- 使用CardLayout的容器可以容纳多个组件，但是实际上同一时刻容器只能从这些组件中选出一个来显示，就像一叠“扑克牌”每次只能显示最上面一张一样，这个被显示的组件将占据所有的容器空间，依次排序。
- CardLayoutTest.java



GridLayout

- GridLayout的布局策略是把容器划分成若干行乘若干列的网格区域，组件就位于这些划分出来的小格中。
- GridLayout比较灵活，划分多少网格由程序自由控制，而且组件定位也比较精确.
- 由于GridLayout布局中每个网格都是相同大小并且强制组件与网格的大小相同，使得容器中的每个组件也都是相同的大小，显得很不自然。为了克服这个缺点，可以使用容器嵌套。
- GridLayoutTest.java



GridBagLayout

- 将组件排列在一行或一列，这取决于创建盒式布局对象时，是否指定了是行排列还是列排列。
- 使用行（列）型盒式布局的容器将组件排列在一行（列），组件按加入的先后顺序从左（上）向右（下）排列，容器的两端是剩余的空间。
- 和FlowLayout布局不同的是，使用行型盒式布局的容器只有一行（列），即使组件再多，也不会延伸到下一行（列），这些组件可能会被缩小大小，紧缩在这一行（列）中。



null布局

- 可以把一个容器的布局设置为null，此时可以通过setSize、setLayout、setBounds等方法准确地定位组件在容器中的位置
- 如：

```
this.setLayout(null);  //空布局
Button btn = new Button("添加");
btn.setBounds(100, 100, 100, 40);
this.add(btn);
```

目　录

1 AWT组件

2 布局管理器

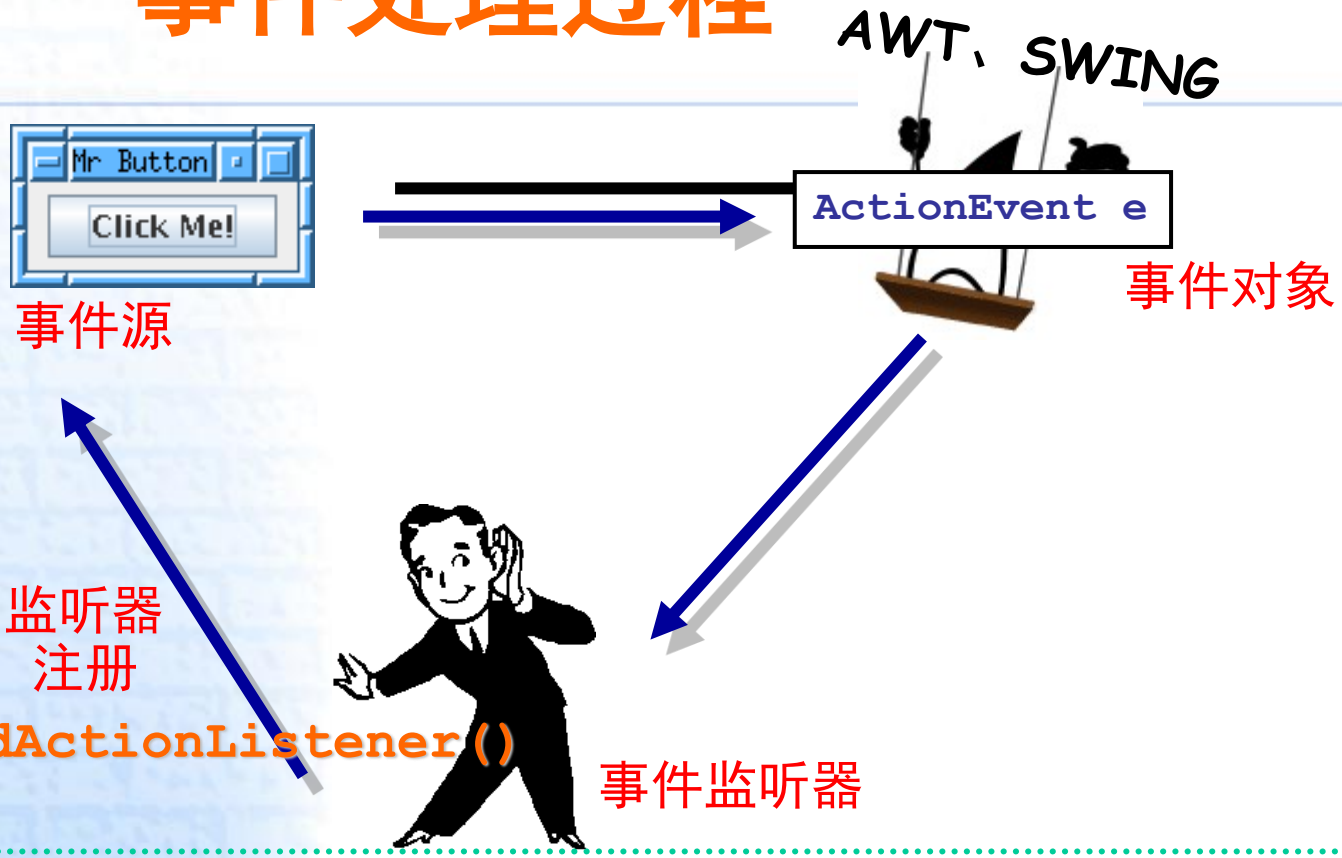
3 Java的事件处理

4 Swing包

Java的事件处理

- 在设计和实现图形用户界面的过程中，主要完成两个任务：
  ① **创建窗口**并在窗口中**添加各种组件**，指定组件的属性和它们在窗口中的位置，从而构成图形界面的外观效果；
  ② 定义图形界面的**事件和各种组件对不同事件的响应**，从而实现图形界面与用户的交互。
- Java 对事件处理采用**委托事件模型**的形式。
  - 组件可以发起（触发）一个事件。
  - 每种事件的类型由单独的类表示。
  - 当事件被触发时，它将被一个或多个“监听器”接收，监听器负责处理事件。

事件处理过程




```
class ButtonListener implements ActionListener {
    public void actionPerformed(                    ) {
        /**   按钮事件所进行的具体工作   **/
    }
}
```

监听-〉报告-〉处理



- 看护小宝宝“换尿布”的方式
  - 传统方法：每过一会就摸摸...
    如果尿布湿了，换新的
  - 新方法：尿布自动报警器(湿敏电阻)

事件和事件源对象

- **事件**
  - 一个描述事件源状态改变的对象 。
  - 通过鼠标、键盘与 GUI 界面直接或间接交互都会生成事件。 如：按下一个按钮、通过键盘输入一个字符、选择列表框中的一项、点击一下鼠标等
- **事件源对象**
  - 是一个生成事件的对象
  - 一个事件源可能会生成不同类型的事件
  - 事件源提供了一组方法，用于为事件注册一个或多个监听器
  - 每种事件的类型都有其自己的注册方法

监听器对象

- 事件监听器，就是一个“实现了某种类型的监听器接口的”类的对象。
  - 编程时，先创建一个监听器对象，然后把它注册给触发事件的组件。这个注册动作是通过调用触发事件的组件的 addXXXListener( ) 方法来完成的，这里用“XXX”表示监听器所监听的事件类型。
  - 通过观察“addListener”方法的名称，就可以很容易地知道其能够处理的事件类型。

按钮单击事件示例

```
public class MyEventTest extends Panel {
    Button myButton = new  Button("蓝色");
      ... // 创建面板或其他 GUI 组件
    MyListener  myAction = new  MyListener();
    MyButton.addActionListener(myAction);
    private class MyListener implements ActionListener {
    public MyListener() {
      . . . // 初始化
    }
    public void actionPerformed(ActionEvent event) {
      . . .//需要为事件进行的操作
    }
  }
}
```

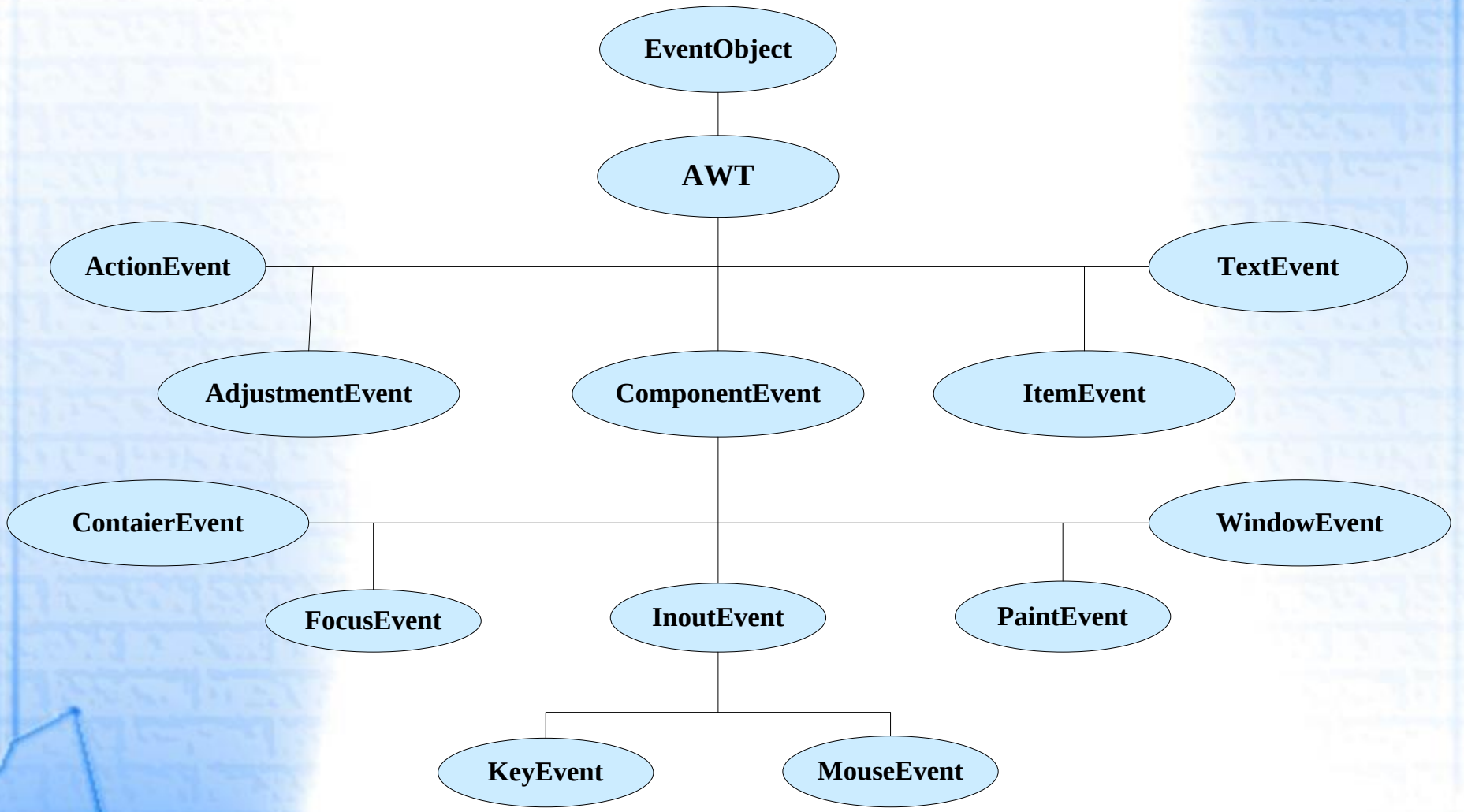
为按钮事件注册监听器

实现了ActionListener接口的监听器

实现了actionPerformed方法，以便根据参数 ActionEvent 对象的信息对事件做出响应

ActionEventTest1.java

事件类的继承关系


EventObject
AWT
ActionEvent
TextEvent
AdjustmentEvent
ComponentEvent
ItemEvent
ContaierEvent
WindowEvent
FocusEvent
InoutEvent
PaintEvent
KeyEvent
MouseEvent

事件的种类

- 按产生事件的物理操作和GUI事件的表现效果进行分类
  - ActionEvent：处理按钮、列表双击、单击菜单项目
  - MouseEvent：处理鼠标拖动、移动、单击、按下、释放或者进入、退出组件的事件
  - WindowEvent：处理窗口激活、失去活动窗口、最小化、打开、关闭或者退出的事件
  - TextEvent：处理文本区域或者文本区域的值得改动
  - ComponentEvent：处理组件被隐藏、移动、尺寸调整或者变为不可见的事件
  - FocusEvent：处理组件获得或失去焦点的事件
  - InputEvent：处理复选框和列表项单击、控件的选择和可选菜单项的选择事件
  - KeyEvent：处理键盘的输入

事件监听器的种类

事件	事件监听器	接口中的主要方法
ActionEvent	ActionListener	actionPerformed()
TextEvent	TextListener	textValueChanged()
ComponentEvent	ComponentListener	ComponentHidded()
		ComponentShown()
		ComponentResized()
		ComponentMoved()
WindowEvent	WindowListener	WindowActivated()
		WindowOpened()
		WindowClosed()
		WindowClosing()
		……

java.nio.channels.spi
java.nio.charset
java.nio.charset.spi
java.nio.file
java.nio.file.attribute
java.nio.file.spi
java.rmi
java.rmi.activation
java.rmi.dgc
java.rmi.registry
java.rmi.server
java.security
java.security.acl
java.security.cert
java.security.interfaces
AWTEvent
AWTEventMulticaster
AWTKeyStroke
AWTPermission
BasicStroke
BorderLayout
BufferCapabilities
BufferCapabilities.FlipContents
Button
Canvas
CardLayout
Checkbox
CheckboxGroup
CheckboxMenuItem

Method Summary

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method and Description
void	**addItemListener**(ItemListener l) Adds the specified item listener to receive item events from this check box.
void	**addNotify**() Creates the peer of the Checkbox.
AccessibleContext	**getAccessibleContext**() Gets the AccessibleContext associated with this Checkbox.
CheckboxGroup	**getCheckboxGroup**() Determines this check box's group.
ItemListener[]	**getItemListeners**() Returns an array of all the item listeners registered on this checkbox.
String	**getLabel**() Gets the label of this check box.
<T extends EventListener>	**getListeners**(Class<T> listenerType)

Packages

java.applet
java.awt
java.awt.color
java.awt.datatransfer
java.awt.dnd
java.awt.event
java.awt.font
java.awt.geom

Classes

AlphaComposite
AWTEvent
AWTEventMulticaster
AWTKeyStroke
AWTPermission
BasicStroke
BorderLayout
BufferCapabilities
BufferCapabilities.FlipContents
Button
Canvas
CardLayout
Checkbox
CheckboxGroup
CheckboxMenuItem
Choice

Method Summary

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method and Description
void	**addActionListener**(ActionListener l) Adds the specified action listener to receive action events from this button.
void	**addNotify**() Creates the peer of the button.
AccessibleContext	**getAccessibleContext**() Gets the AccessibleContext associated with this Button.
String	**getActionCommand**() Returns the command name of the action event fired by this button.
ActionListener[]	**getActionListeners**() Returns an array of all the action listeners registered on this button.
String	**getLabel**() Gets the label of this button.

适配器

- 实现了相应接口的类：为接口中的方法添加了方法体
- class MyListener **implements** WindowListener
  {
    要实现的方法：windowActivated()、windowClosed()、windowClosing()、 windowDeactivated windowDeiconified()、 windowIconified()、windowOpened()
- }
- class MyListener **extends WindowAdapter**
  {
    只要实现需要重写的方法即可。
- }

Example02.java

事件监听器的实现形式

- 事件监听器是一个特殊的java对象，实现事件监听器对象有如下几种形式：
  - 顶级类形式：将事件监听器类定义成一个顶级类
    - 不能自由访问创建GUI界面的类中的组件
    - 可以被多个GUI界面所共享，完成某种业务逻辑的实现
  - 类本身作为事件监听器类：让当前类本身实现监听器接口或继承事件适配器(Example03.java; SimpleEventHandle.java)
  - 匿名内部类的形式：使用匿名内部类创建事件监听器对象(Example04.java; AnonymousEventHandle.java)

事件处理例1

- CardLayoutTest.java



目　录

1 AWT组件

2 布局管理器

3 Java的事件处理

4 Swing包

swing包

- 为了克服AWT包中隐含的图形组件，在不同的操作平台上显示不同效果问题，1998年推出的JDK1.2版本时，将javax.swing包列入java基础库。
- AWT与swing包的比较
  - swing 包拥有4倍于AWT的用户界面组件
  - 新版的swing包可能与旧版不兼容
  - swing包建立在AWT的基础上，不大可能完全舍弃AWT
  - AWT包的运行速度比swing包快
  - 将AWT包改写为swing包时，在很多情况下可以只在原有的AWT组件的每个类名前面加上“J”即可。

详细内容见
软件包 javax.swing

创建JFrame窗口

JFrame类的构造方法	主要功能
JFrame()	创建没有标题的窗口
JFrame(String title)	创建以title 为标题的窗口
JFrame类的方法	**主要功能**
Container getContentPane()	获得窗口的ContentPane组件
int getDefaultCloseOperation()	当用户关闭窗口时的默认处理方法
int setDefaultCloseOperation()	设置用户关闭窗口时发生的操作
void update(Graphics g)	引用paint()方法重绘窗口
void remove(Component component)	将窗口中指定的组件删除
JMenuBar getMenuBar()	获得窗口中的菜单栏组件
void setLayout(LayoutManager manager)	设置窗口的布局

创建JFrame容器时注意

- 创建的JFrame 窗口是不可见的，要使得可见，需要 setVisible(boolean b)方法，其中setVisible中的参数b=true。
- Swing窗体可以通过调用 setDefaultCloseOperation(参数);来关闭窗口，参数如下：

  DO_NOTHING_ON_CLOSE：不执行任何操作；
  HIDE_ON_CLOSE：自动隐藏该窗体，默认设置；
  DISPOSE_ON_CLOSE：自动隐藏并释放该窗体；
  EXIT_ON_CLOSE：退出应用程序。

- 将按钮组件加入到窗口的一般操作方法：

  1、设置窗口的ContentPane，创建按钮对象

  ```
  Container  c = getContentPane();
  but1=new JButton(“ 按钮1”);
  ```

  2、添加按钮组件到JFrame窗口中

  ```
  c.add(but1);
  ```

Swing例子

- 创建带有JLabel标签和JButton按钮的JFrame窗体。
  - JLabel标签中含有文字 和 图标Icon
  - 按下JButton时触发事件，弹出消息对话框
- jframetest.java



BoxLayout 和 Box容器

- BoxLayout位于javax.swing包中
- BoxLayout 把控件依次进行水平或者垂直排列布局，通过参数 X_AXIS（表示水平排列）、Y_AXIS（表示垂直排列） 来确定。
  - **JPanel panel=new JPanel();**
  - **BoxLayout layout=new BoxLayout(panel, BoxLayout.X_AXIS);**
  - **panel.setLayout(layout);**
- BoxLayout 往往和 Box 容器结合在一起使用
- Box 类使用BoxLayout作为其默认布局，并且定义了4种透明的组件作为填充来填充控件之间的空间，从而达到调整控件之间间隔的目的。

BoxLayout 和 Box容器

- Box 容器提供了 3 种透明的组件，分别是 rigid area、strut、glue
- Rigid area 是一种用户可以定义水平和垂直尺寸的透明组件；

  如： Box.createRigidArea(new Dimension(10, 10));
- strut 与 rigid area 类似，但是用户只能定义一个方向的尺寸，即水平方向或者垂直方向，不能同时定义水平和垂直尺寸；

  如： Box.createHorizontalStrut(10);

  Box.createVerticalStrut(10);
- 当用户将 glue 放在两个控件之间时，它会尽可能的占据两个控件之间的多余空间，从而将两个控件挤到两边；

  如： Box.createGlue();

  Box.createVerticalGlue();

  Box.createHorizontalGlue();

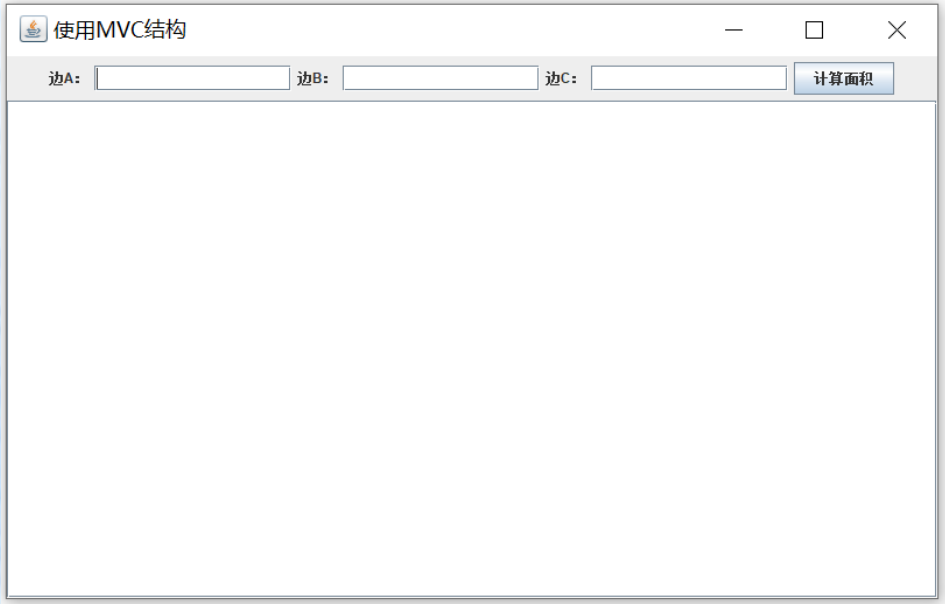
BoxLayoutTest.java

MVC模式

- 一种设计模式，模型-视图-控制器（Model-view-controller）
- 使用MVC的目的是将M和V的实现代码分离，从而使同一个程序可以使用不同的表现形式。其中，View的定义比较清晰，就是用户界面。M表示业务规则，C存在的目的则是确保M和V的同步，一旦M改变，V应该同步更新。
- MVC是Xerox PARC在二十世纪八十年代为编程语言Smalltalk－80发明的一种软件设计模式，已被广泛使用。后来被推荐为Oracle旗下Sun公司Java EE平台的设计模式，并且受到越来越多的使用ColdFusion和PHP的开发者的欢迎。

示 例

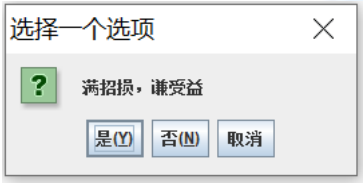
- 利用MVC模式完成三角形面积的计算，三角形的三边通过View来进行输入
  - 内部三角形结构——Model
  - 视图上的内容——View
  - “计算面积”——Controller



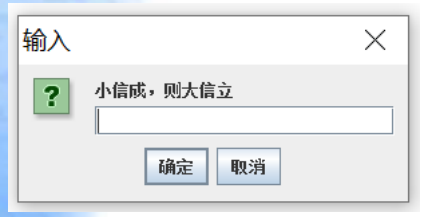
TestMVC.java

JOptionPane

- 主要用到四种消息提示框方法
  - showConfirmDialog()：确认对话框
  - showInputDialog()：输入对话框
  - showMessageDialog()：消息对话框
  - showOptionDialog()：选择对话框



- 示例
  - JOptionPane.showConfirmDialog(this, "满招损， 谦受益");
  - JOptionPane.showInputDialog(this,"小信成， 则大信立");
  - JOptionPane.showMessageDialog(null, “错误” ,”提示” ,JOptionPane.ERROR_MESSAGE);





绘制图形

- java.awt包中的Graphics、 Graphics2D类
- Graphics类是所有图形上下文的抽象基类，提供基本的几何图形绘制方法，主要有：画线段、画矩形、画圆、画带颜色的图形、画椭圆、画圆弧、画多边形等。
  - 如：Graphics类的drawLine()方法：drawLine(int x1,int y1,int x2,int y2)，在此图形上下文的坐标系中，使用当前颜色在点 (x1,y1) 和 (x2,y2)之间画一条线。
- Graphics2D 类扩展 Graphics 类，以提供对几何形状、坐标转换、颜色管理和文本布局更为复杂的控制。它是用于在 Java平台上呈现二维形状、文本和图像的基础类。
- java.awt.geom包里有很多基本形状类，如：Arc2D、Ellipse2D、Rectangle2D等

一些概念

- 图形上下文：通俗点讲，就是画图环境。每个窗口构件（如主窗口、按钮等），都有一个自己的图形上下文对象，我们就是使用这个对象来实现在构件上画图。这个对象就是Graphics对象。
- 如何获得图形上下文：我们要在哪个构件上绘图，就调用那个构件的getGraphics()方法即可获得该构件的图形上下文对象，然后使用这个对象绘图。
- Java坐标系：Java的坐标原点（0，0）位于屏幕的左上角，坐标度量以象素为单位，水平向右为X轴的正方向，竖直向下为Y轴的正方向，每个坐标点的值表示屏幕上的一个像素点的位置，所有坐标点的值都取整数。
- 当前颜色：指图形上下文当前的颜色。每个图形上下文都有自己当前的颜色。通过Graphics对象的getColor()方法可获取该颜色，setColor()方法可设置颜色。

重写paint方法

- Component类有一个方法public void paint(Graphics g)，程序可以在其子类中重写这个方法。
- 当程序运行时，java运行环境会用Graphicd2D（Graphics的一个子类）将参数g实例化，对象g就可以在重写paint方法的组件上内绘制图形、图像等，Graphics2D的“画笔”分别使用draw和fill方法来绘制和填充一个图形。

```
public void paint(Graphics g)
{
    Graphics2D g_2d = (Graphics2D)g;
    Arc2D arc = new Arc2D.Double(100, 100, 100, 100, 0, -180, Arc2D.PIE);
    g_2d.setColor(Color.BLACK);
    g_2d.fill(arc);
    g_2d.setColor(Color.white);
    arc.setArc(100, 100, 100, 100, 0, 180, Arc2D.PIE);
    g_2d.fill(arc);

    g_2d.setColor(Color.red);
    Ellipse2D ellipse = new Ellipse2D.Double(140, 115, 20, 20);
    g_2d.fill(ellipse);

    g_2d.setColor(Color.yellow);
    arc.setArc(125, 150, 50, 50, 90, -180, Arc2D.PIE);
    g_2d.fill(arc);

    g_2d.setColor(Color.blue);
    ellipse.setFrame(140,165, 20, 20);
    g_2d.fill(ellipse);
```



TestDraw.java

本章总结

- 本章我们了学习Java语言的用户界面的设计，及如何进行友好方便的人机交互。
  - Java中AWT包
  - 布局管理器的概念和使用
  - Java的事件处理机制
  - Swing包的简介

Class is Dismissed!

